CSE - 494 INTELLIENT NOSQL DATABASES

PROJECT REPORT

(Project Semester August - Decmeber 2025)

GYM MANAGEMENT SYSTEM

Submitted by:

Name of student: Ponnuru Nithin & Badday Hemanth Reddy

Registration No: 12303486 & 12307348

Programme and Section: RK2P23EZA23

Course Code: CSE - 494

Submitted to:

Name of faculty: Ankit Galhotra

U.Id: 32995



LOVELY PROFESSIONAL UNIVERSITY

School of Computer Science Engineering

Lovely Professional University, Phagwara

Certificate

This is to certify that Nithin Ponnuru & Badday Hemanth Reddy bearing Registration no. 12303486 & 12307348 has completed CSE - 494 project titled Gym Management System my guidance and supervision. To the best of my knowledge, the present work is the result of his/her original development, effort and study.

Name of the Supervisor: Ankit Galhotra

Signature: Ankit Galhotra

Designation of the Supervisor: No sql Professor

School of Computer Science Engineering

Lovely Professional University

Phagwara, Punjab.

Date: 24-10-2026

Declaration

I, Nithin Ponnuru & Badday Hemanth Reddy, student of under School of Computer Science Engineering CSE/IT Discipline at, Lovely Professional University, Punjab, hereby declare that all the information furnished in this project report is based on my own intensive work and is genuine.

Name of the student: Nithin Ponnuru & Badday Hemanth Reddy

Registration No: 12303486 & 12307348

Date: 24-10-202

Signature: P. Nithin & B.Hemanth

Acknowledgement

We express our sincere gratitude to our supervisor, Ankit Galhotra, and the Department of Computer Science and Engineering, Lovely Professional University, for their mentorship throughout the development of the FitPro Gym Management System. We thank the university leadership for providing high-performance computing facilities and our peers for constructive critiques. We also acknowledge the open-source communities behind Node.js, Express.js, React.js, and MongoDB for their robust tooling and documentation.

Abstract

The FitPro Gym Management System is a full-stack web application developed using the MERN stack to streamline gym operations, including member management, trainer scheduling, membership subscriptions, and attendance tracking. It features separate Super Admin and Member portals, with MongoDB 7.0 and Mongoose managing eight dedicated collections using optimized schemas and indexing to ensure data consistency and prevent duplicate daily check-ins. The backend incorporates real-time notifications using Socket.IO, automated membership expiry alerts using Node-Cron, and MongoDB aggregation pipelines for revenue and attendance analytics. The frontend uses React 18, Vite, and Tailwind CSS, along with an integrated FitAI assistant powered by OpenRouter. Testing demonstrates reliable attendance deduplication, sub-second query performance, and persistent notifications. The system improves operational efficiency, reduces manual errors, and provides administrators with centralized control over gym activities. It also offers members a convenient platform to access membership information and receive timely updates. The application follows a modular architecture that supports maintainability, scalability, and future feature development. Secure role-based access helps ensure that administrators and members can access only their authorized functionalities. Overall, FitPro provides a practical digital solution for modern gym management while creating opportunities for future automation and enhanced member engagement. Future enhancements include payment gateway intergration and biometrics attendance systems.

Keywords: MERN Stack, MongoDB, Express.js, React.js, Node.js, Gym Management System, Database Aggregation, Document Modeling.

Table of Contents

Certificate ..... **ii**

Declaration ..... **ii**

Acknowledgement ..... **iii**

Abstract ..... **iii**

Chapter 1: Introduction **1**

1.1 Project Overview 1

1.2 Background and Need 1

1.3 Scope 1

1.4 Project Objectives 2

Chapter 2: Problem Statement and Proposed Solution **3**

2.1 Problem Statement 3

2.2 Existing Approach and Limitations 3

2.3 Proposed Solution 3

Chapter 3: Requirement Analysis **4**

3.1 Functional Requirements 4

3.2 Non-functional Requirements 4

3.3 Hardware Requirements 5

3.4 Software and Technology Stack 5

Chapter 4: System Design and Architecture **6**

4.1 System Architecture 6

4.2 Module Description 6

4.3 Workflow / Use-case Diagram 7

4.4 Security and Access 7

Chapter 5: Database Design Using MongoDB **8**

5.1 Database and Collection Design 8

5.2 Document Structure and Field Types 9

5.3 Sample Documents 12

5.4 Relationships: Embedding or Referencing 12

5.5 Validation Rules 12
5.6 Indexes 13
Chapter 6: Implementation 14
6.1 Application Setup and Database Connection 14
6.2 CRUD Operations 14
6.3 Querying, Sorting and Projection 15
6.4 Aggregation Pipeline 15
6.5 Website Screenshots: Authentication & Admin Operations 15
Chapter 7: Testing and Results 21
7.1 Testing Approach 21
7.2 Test Cases 21
7.3 Results and Discussion 22
7.4 Limitations 22
Chapter 8: Conclusion and Future Scope 23
8.1 Conclusion 23
8.2 Future Enhancements 23
References 24
Appendix A: Selected Code Excerpts 25
Appendix B: Supplementary MongoDB Compass Database Evidence 25
Appendix C: Setup & Execution Instructions ..... Error! Bookmark not defined.

Key Figures and Abbreviations Summary

Figures: Fig 4.1 System Architecture; Fig 4.2 Workflow Flowchart; Fig 5.1 Compass Collections; Fig 5.2 Compound Index; Figs 6.1–6.11 Website Interfaces; Figs B.1–B.4 Compass Collections.

Abbreviations: API, BSON, CRUD, CSS, DOM, GUI, HTTP, JSON, JWT, KPI, LLM, LPU, MERN, MVC, ODM, RBAC, REST, SPA, UI, URI, WOD.

Chapter 1: Introduction

1.1 Project Overview

The FitPro Gym Management System is a comprehensive, enterprise-ready web application engineered to digitize, streamline, and automate the operational lifecycle of contemporary health and fitness centers. Developed on the MERN stack (MongoDB, Express.js, React.js, Node.js), FitPro enforces a strict dual-portal architecture separating Administrative Oversight from Member Self-Service:

- **Admin Portal** — Provides gym managers with real-time KPI dashboards, full CRUD for members and trainers, membership plan catalog controls, attendance verification, automated expiration tracking, and an 8-category notification center.
- **Member Portal** — Allows clients to view active subscription badges, days remaining, browse membership packages, submit purchase/renewal requests, inspect check-in histories, and consult FitAI.

1.2 Background and Need

Independent and mid-sized fitness centers often rely on paper logbooks, disconnected spreadsheets, and informal messaging. This introduces critical operational challenges:

- **Revenue Leakage via Expiry Blindspots** — Staff overlook expiring memberships, allowing members to use facilities past paid tenures.
- **Attendance Fraud & Proxy Logging** — Manual sign-in sheets allow proxy check-ins; spreadsheets lack constraints, resulting in duplicate entries.
- **Trainer Caseload Imbalance** — Managers lack live metrics on member distribution across certified trainers.
- **Communication Delays** — Members lack visibility into request approvals and renewal dates, overwhelming staff with inquiries.

1.3 Scope

The implemented scope comprises JWT role-based access control, 8 production MongoDB collections, automated sequential ID generation (MEM# and TRN#), subscription approval workflows with embedded plan snapshots, atomic check-in deduplication via compound unique indexes, real-time Socket.IO notification push, daily Node-Cron expiration sweeps at

08:00 AM, and an embedded LLM-driven FitAI assistant. Planned future boundaries include automated third-party payment gateway webhooks (Razorpay/Stripe) and physical biometric turnstile hardware.

1.4 Project Objectives

The project was engineered to satisfy five verifiable technical objectives:

- **Objective 1: Optimized Schema Design** — Design a normalized database architecture across 8 MongoDB collections utilizing BSON types and hybrid embedding for historical subscription snapshots.
- **Objective 2: Database-Level Attendance Integrity** — Enforce zero duplicate daily check-ins via compound unique indexing ({ memberId: 1, date: 1 }), rejecting duplicate entries with HTTP 409.
- **Objective 3: Real-Time Event Notifications** — Implement a Socket.IO WebSocket engine broadcasting push alerts across 8 categorized filter tabs upon member registration, requests, and trainer additions.
- **Objective 4: Automated Expiration Auditing** — Deploy an autonomous Node-Cron daemon executing daily at 08:00 AM to detect subscriptions expiring within 7 days with duplicate alert suppression.
- **Objective 5: High-Performance Aggregations** — Execute multi-stage MongoDB aggregations ($match, $lookup, $unwind, $group, $sort) for revenue and attendance analytics natively with sub-second execution.

Chapter 2: Problem Statement and Proposed Solution

2.1 Problem Statement

Fitness centers experience continuous footfall, multi-tier subscriptions, and frequent trainer allocations. Traditional manual registers or standalone spreadsheets fail to enforce relational validation, transactional atomicity, or automated alerting, resulting in inconsistent records, untracked subscription expirations, proxy attendance, and administrative inefficiency.

2.2 Existing Approach and Limitations

Common existing practices include paper registers and shared Excel sheets. Paper logs suffer from physical wear, data loss, and zero searchability. Spreadsheets lack role-based access control, foreign key constraints, automated schedulers, and real-time WebSockets, frequently suffering data overwrites when accessed concurrently.

2.3 Proposed Solution

FitPro centralizes gym operations through a modern full-stack web architecture. Operational data resides in MongoDB (GymManagementDB). Admins and members authenticate via JWT tokens, routing to dedicated interfaces. Socket.IO rooms push real-time alerts upon state changes, while background cron daemons automate subscription expiry audits, and native aggregations power live executive dashboards.

Chapter 3: Requirement Analysis

3.1 Functional Requirements

Table 3.1 details the verified functional requirements implemented across all application modules.

Table 3.1: Functional Requirements Specification

Req ID	Module	Functional Requirement Description	Status
FR-01	Authentication	Authenticate Admins and Members using JWT bearer tokens; verify credentials via bcryptjs hashes; return HTTP 401 on failure.	Verified
FR-02	Members	Perform CRUD operations on members; auto-generate sequential MEM# IDs via pre-save hooks; support multi-field regex search.	Verified
FR-03	Trainers	Manage trainers across 10 disciplines; calculate caseload counts via aggregation; cascade null to members on deletion.	Verified
FR-04	Memberships	Maintain catalog tiers (Basic, Standard, Premium, Gold) with duration months, price, feature arrays, and highlight badges.	Verified
FR-05	Attendance	Record daily check-ins; enforce zero duplicate check-ins per member per day via compound unique index ({ memberId: 1, date: 1 }).	Verified
FR-06	Workouts	Create routines with embedded exercise sub-documents (sets, reps, rest intervals) categorized by difficulty and goal.	Verified
FR-07	Notifications	Dispatch Socket.IO alerts across 8 filter tabs upon member registration, plan requests, approvals, and trainer additions.	Verified
FR-08	FitAI Assistant	Provide interactive conversational AI integrating OpenRouter API to generate customized training and diet plans.	Verified

3.2 Non-functional Requirements

- **Security** — Passwords hashed via bcryptjs (10 salt rounds); JWT tokens signed with HMAC SHA-256 (24h lifespan); sanitized inputs; masked error stack traces.
- **Data Integrity** — Mongoose schema types, required constraints, enums, compound unique indexes, and referential cleanup hooks prevent dangling pointers.

- **Performance** — Indexed MongoDB lookups and native aggregations achieve sub-second execution; Socket.IO alert delivery under 15ms.
- **Reliability** — 100% database persistence in MongoDB; notifications and subscriptions remain intact across server restarts and client reloads.

3.3 Hardware Requirements

- **Server / Dev Workstation** — x86-64 quad-core CPU 2.4 GHz, 8 GB RAM, 500 MB SSD storage, local TCP network.
- **Client Machine** — Dual-core 1.8 GHz CPU, 4 GB RAM, 1280x720 display, modern evergreen browser (Chrome, Edge, Firefox).

3.4 Software and Technology Stack

Table 3.2: Software and Technology Stack Version Matrix

Layer / Component	Technology	Verified Version	Architectural Function
Database Engine	MongoDB Server	v7.0 Community	BSON document store supporting compound indexing and aggregations.
Database Modeling	Mongoose ODM	v9.10 / v7.8	Object data modeling, schema validation, pre-save hooks, and population.
Server Runtime	Node.js	v24.x LTS	Asynchronous JavaScript runtime executing Express and background cron jobs.
Backend API	Express.js	v4.22.2	RESTful routing, authentication middleware, and error handling.
Real-Time Engine	Socket.IO	v4.8.4	Bidirectional WebSocket broadcasting push alerts to the admin room.
Scheduler	Node-Cron	v4.6.0	Automated cron daemon running daily 08:00 AM membership expiry audits.
Client Framework	React.js	v19.2 / v18.x	Component-driven single page application managing interactive state.
Build Tool	Vite	v8.3 / v6.x	Frontend build tool providing Hot Module Replacement and production bundling.
Styling & Charts	Tailwind CSS & Recharts	v4.3 / v3.10	Utility styling framework and SVG chart rendering for dashboard analytics.

Chapter 4: System Design and Architecture

4.1 System Architecture

The FitPro architecture follows a decoupled, multi-tier client-server structure across Presentation (React SPA), Application/Services (Node.js + Express REST API & Socket.IO), Data Modeling (Mongoose ODM), and Persistence (MongoDB 7.0). Figure 4.1 illustrates the architectural components and data communication flows.

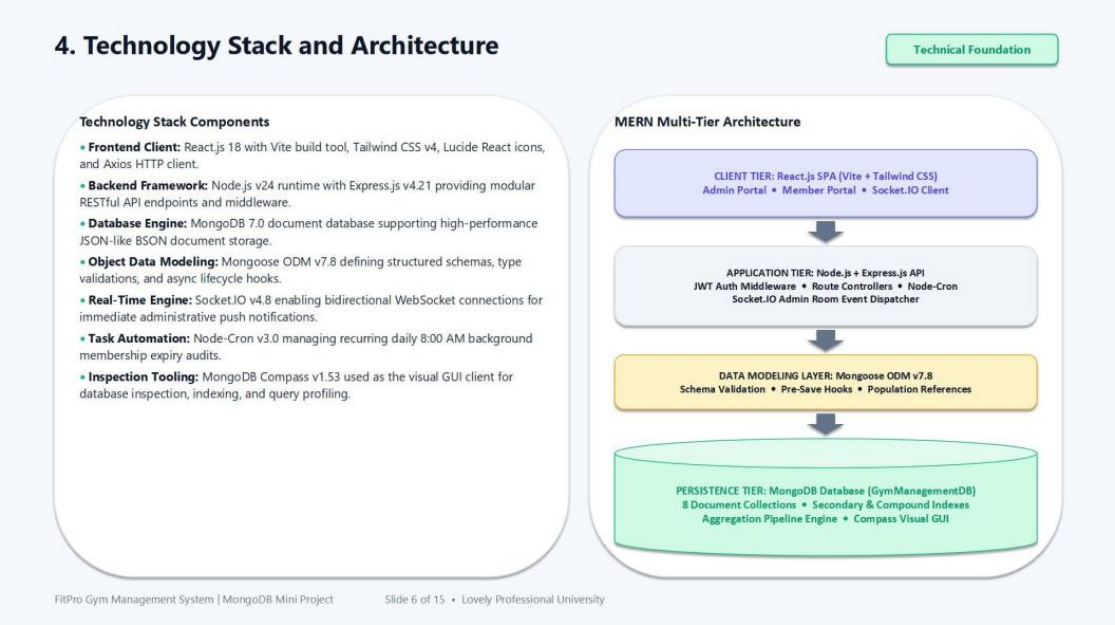


Figure 4.1: FitPro Multi-Tier System Architecture Diagram

4.2 Module Description

1. **Executive Dashboard** — Computes total members, active counts, trainer numbers, total revenue, and expiring plans via MongoDB aggregations.
2. **Member Management** — Provides registration, automated MEM# ID generation, multi-field regex search, and assigned trainer updates.
3. **Trainer Directory** — Maintains profiles across 10 fitness specializations with real-time caseload counts and safe deletion cascades.
4. **Memberships Catalog** — Defines plan tiers, duration, pricing, and promotional badges; embeds immutable plan snapshots upon purchase.
5. **Request Approval Queue** — Manages member purchase requests; admins audit and activate plans, triggering Socket.IO notifications.

6. **Attendance Registry** — Logs daily arrivals with timestamping; enforced by compound unique index to physically prevent duplicates.
7. **Notification Engine** — Pushes real-time alerts across 8 filter categories; synchronizes unread badges persistently in MongoDB.

4.3 Workflow / Use-case Diagram

Figure 4.2 illustrates the end-to-end execution flow: credential authentication, JWT role verification, portal branching, API gateway processing, MongoDB execution, and Socket.IO broadcast.

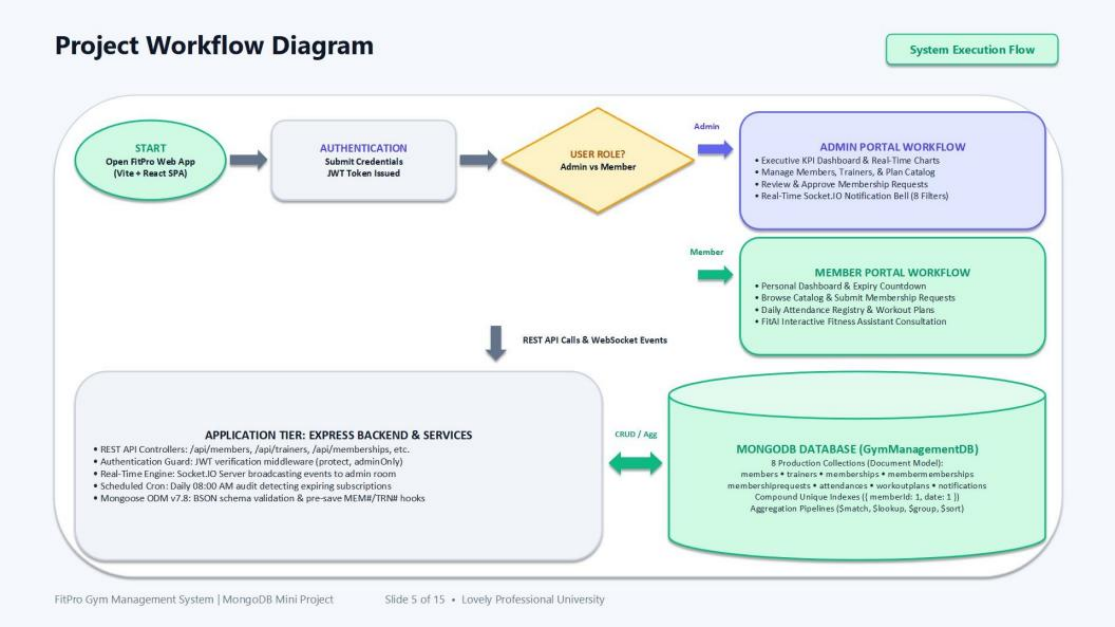


Figure 4.2: FitPro End-to-End System Execution Flowchart

4.4 Security and Access

Security is enforced through defense-in-depth: Role-Based Access Control (RBAC) via protect and adminOnly middleware; stateless HMAC SHA-256 signed JWT tokens (24h lifespan); passwords hashed via bcryptjs with 10 salt rounds; strict CORS origin whitelisting; and sensitive database/API secrets isolated in .env.

Chapter 5: Database Design Using MongoDB

5.1 Database and Collection Design

The persistence layer is implemented in MongoDB 7.0 Community Server under the database identifier GymManagementDB. Table 5.1 itemizes the 8 collections and their document roles.

Table 5.1: MongoDB Collection Inventory (GymManagementDB)

Collection	Mongoose Model	Entity Description	Index & Optimization
members	Member	Registered gym members, credentials, contact info, assigned trainer.	Indexed on email, memberId, status, and text search.
trainers	Trainer	Certified coaches across 10 disciplines with experience and salary.	Indexed on email, specialization, and employment status.
memberships	Membership	Master plan catalog defining duration months, price, and features.	Indexed on unique planName and price sorting.
membermemberships	MemberMembership	Active and historical subscriptions with embedded plan snapshots.	Indexed on memberId, status, and expiryDate for cron scans.
membershiprequests	MembershipRequest	Member purchase requests pending administrative approval.	Indexed on memberId, status, and createdAt timestamp.
attendances	Attendance	Daily check-in registry with member IDs, timestamps, and status.	Compound unique index ({ memberId: 1, date: 1 }).
workoutplans	WorkoutPlan	Workout regimens with embedded exercise sub-documents.	Indexed on planName (text), goal, and trainerId.
notifications	Notification	Administrative alerts with event type enums, read flags, metadata.	Indexed on type, isRead, and createdAt for tab filtering.

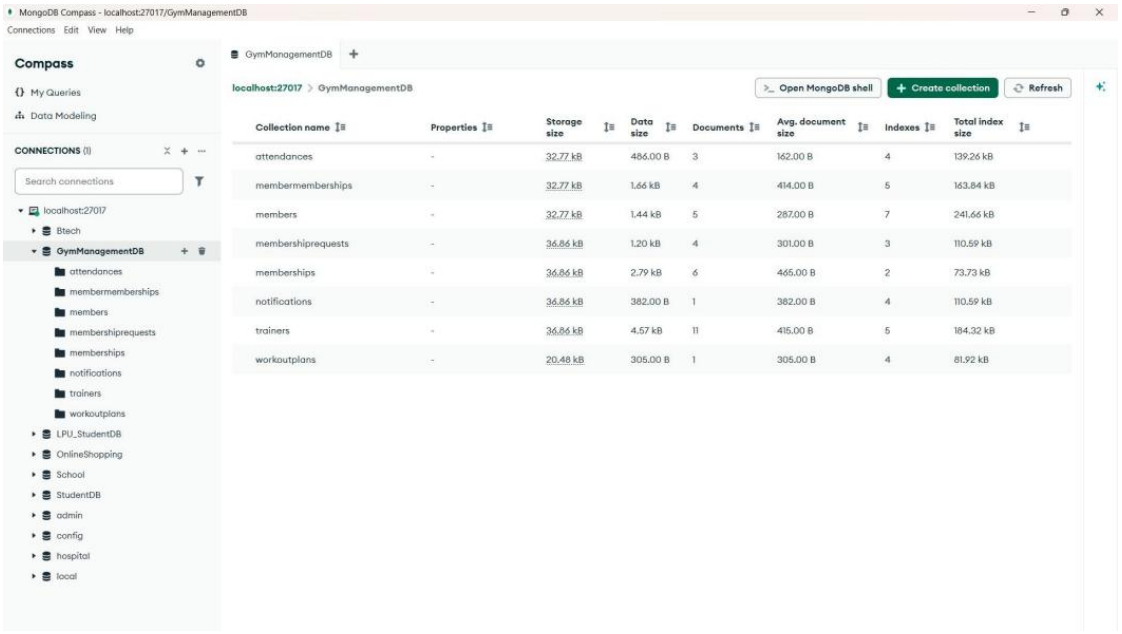


Figure 5.1: MongoDB Compass Database Collection Inventory (GymManagementDB)

5.2 Document Structure and Field Types

Tables 5.2 through 5.5 document field dictionaries for core application collections.

Table 5.2: Field Dictionary: 'members' Collection

Field	Type	Constraint	Default	Purpose & Validation
_id	ObjectId	Primary Key	Auto-gen	Unique 12-byte BSON identifier.
memberId	String	Unique, Req	MEM0001 (Hook)	Sequential ID generated pre-save.
name	String	Required	None	Full member name, trimmed.
email	String	Unique, Req	None	Validated via email regex.
password	String	Required	None	Bcryptjs hashed password.
phone	String	Required	None	Contact phone (7-15 digits).
trainerId	ObjectId	Ref: Trainer	null	Reference to assigned trainer.
status	String	Enum	'Active'	Status: ['Active', 'Inactive'].

Table 5.3: Field Dictionary: 'trainers' Collection

Field	Type	Constraint	Default	Purpose & Validation
_id / trainerId	ObjectId / String	Primary / Unique	TRN001 (Hook)	Unique ID & sequential identifier.
name / email / phone	String	Required, Unique	None	Trainer credentials and contact info.
specialization	String	Enum, Required	None	10 Disciplines: Weight, Yoga, CrossFit, etc.
experience / salary	Number	Min: 0	None	Years coaching experience and INR salary.

Table 5.4: Field Dictionary: 'membermemberships' Collection

Field	Type	Constraint	Default	Purpose & Validation
_id	ObjectId	Primary Key	Auto-gen	Unique 12-byte BSON identifier.
memberId	ObjectId	Ref: Member, Req	None	Reference to subscriber.
membershipId	ObjectId	Ref: Membership	None	Reference to master plan.
startDate / expiryDate	Date	Required	Date.now / calc	Subscription validity boundary.
amountPaid	Number	Required, Min: 0	None	Transacted currency in INR.
paymentStatus	String	Enum	'Paid'	Status: ['Paid', 'Pending', 'Partial'].
status	String	Enum	'Active'	Status: ['Active', 'Expired', 'Upcoming'].
planSnapshot	Sub-Doc	Optional	{}	Embedded immutable snapshot of plan terms.

Table 5.5: Field Dictionary: 'attendances' Collection

Field	Type	Constraint	Default	Purpose & Validation
_id	ObjectId	Primary Key	Auto-gen	Unique 12-byte BSON identifier.
memberId	ObjectId	Ref: Member, Req	None	Part of compound unique index.
date	Date	Required	None	Midnight timestamp; part of compound index.
checkInTime	String	Optional	HH:MM	Time recorded upon gym arrival.
status	String	Enum	'Present'	Status: ['Present', 'Absent'].

Table 5.6: Field Dictionary: 'membershiprequests' Collection

Field	Type	Constraint	Default	Purpose & Validation
_id	ObjectId	Primary Key	Auto-gen	Unique 12-byte BSON identifier.
memberId / membershipId	ObjectId	Ref: Member / Plan	None	References to requesting member and catalog plan.
requestedStartDate	Date	Required	Date.now	Member's preferred start date.
status	String	Enum	'Pending'	Workflow status: ['Pending', 'Approved', 'Rejected'].
paymentMethod	String	Enum	'Offline'	Method: ['Offline', 'Cash', 'UPI', 'Card'].

Table 5.7: Field Dictionary: 'workoutplans' Collection

Field	Type	Constraint	Default	Purpose & Validation
_id / planName	ObjectId / String	Primary / Required	None	Unique ID & descriptive routine title.
goal / difficulty	String	Enum, Required	None	Goal: Weight Loss, Muscle, etc. Difficulty: Beginner, Adv.
trainerId	ObjectId	Ref: Trainer	null	Supervising coach reference.
exercises	Array of Sub-Docs	Min: 1 exercise	[]	Embedded: [{ name, sets, reps, restSeconds }].

Table 5.8: Field Dictionary: 'notifications' Collection

Field	Type	Constraint	Default	Purpose & Validation
_id / type	ObjectId / String	Primary / Enum	None	Unique ID & event enum: NEW_MEMBER, PLAN_REQUEST, etc.
title / message	String	Required	None	Headline and detailed body text.
isRead / priority	Boolean / String	Enum	false / 'normal'	Read flag & priority: low, normal, high, urgent.
metadata	Mixed Object	Optional	{}	Stores refId, memberId, and reminderKey for deduplication.

5.3 Sample Documents

```
// Sample BSON Document with Embedded Plan Snapshot
// Sample Document: membermemberships (Embedded Snapshot)
{
"_id": "64f1a2b3c4d5e6f7a8b9c0e2", "memberId": "64f1a2b3c4d5e6f7a8b9c0d1",
"membershipId": "64f1a2b3c4d5e6f7a8b9c030", "startDate": "2026-09-
20T00:00:00.000Z", "expiryDate": "2026-12-20T23:59:59.999Z",
"amountPaid": 2499, "paymentStatus": "Paid", "status": "Active",
"planSnapshot": { "planName": "Standard", "durationMonths": 3, "price": 2499,
"features": ["Gym Access", "Locker Room"] }
}
```

5.4 Relationships: Embedding or Referencing

FitPro applies a hybrid relational model: normalized ObjectId references link members to trainers and subscriptions to members, ensuring profile updates propagate without duplication. Conversely, planSnapshot is embedded in MemberMembership to freeze pricing terms at purchase time, protecting historical financial audits against catalog changes.

5.5 Validation Rules

Validation enforces email regex, age limits (5-120), positive pricing (min: 0), and discipline enums. Submitting a registration with an invalid email ('user.domain') terminates before database insertion and returns HTTP 400 Bad Request with: { success: false, message: 'Validation Error', errors: ['Please enter a valid email'] }.

5.6 Indexes

To guarantee high throughput, primary keys, email, memberId, and text search fields are indexed. Crucially, attendances enforces the compound unique index memberId_1_date_1, physically prohibiting duplicate daily check-ins.

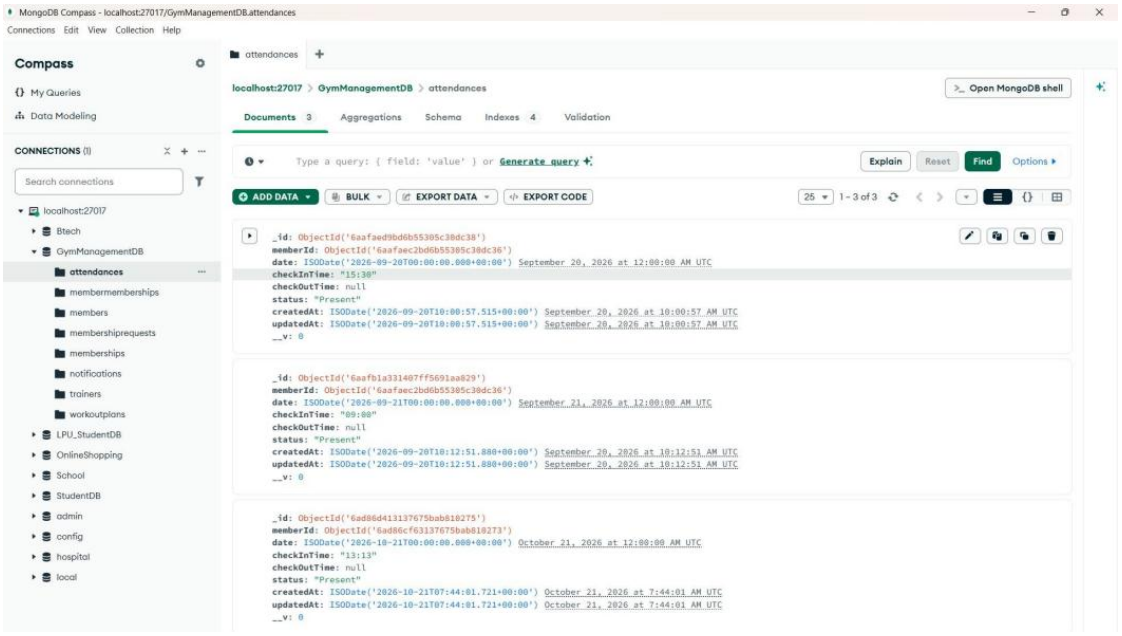


Figure 5.2: MongoDB Compass Compound Unique Index Inspection on 'attendances'

Chapter 6: Implementation

6.1 Application Setup and Database Connection

The server initializes via Mongoose ODM in server/config/db.js, connecting to GymManagementDB with retry logic. Configuration variables (.env) define PORT=5000, MONGODB_URI, and JWT_SECRET without exposing credentials.

6.2 CRUD Operations

Representative CRUD implementations from backend controllers:

```
// Core CRUD Controller Excerpts
// Create: Auto-generate MEM# ID and save member (memberController.js) const
member = await Member.create(req.body);
await member.populate("trainerId", "name specialization");
await createNotification({ type: "NEW_MEMBER", title: "New Member Registered",
message:
`${member.name} registered.` });

// Read: Search with regex $or across 4 fields with pagination
(memberController.js)
const query = { $or: [{ name: { $regex: search, $options: "i" } }, { email:
{ $regex: search,
$options: "i" } }] };
const [members, total] = await
Promise.all([Member.find(query).skip(skip).limit(limit).lean(),
Member.countDocuments(query)]);

// Update: Approve plan request and activate subscription
(membershipRequestController.js) request.status = "Approved"; await
request.save();
const sub = await MemberMembership.create({ memberId, membershipId, startDate,
expiryDate, planSnapshot: { planName, price } });

// Delete: Cascade unassign trainer from members (trainerController.js) await
Trainer.findByIdAndDelete(req.params.id);
await Member.updateMany({ trainerId: req.params.id }, { $set: { trainerId:
null } });
```

6.3 Querying, Sorting and Projection

The attendance module queries records within a calendar day range ({ $gte: startOfDay, $lt: endOfDay }) using lean projections.

6.4 Aggregation Pipeline

To compute Revenue by Plan tier on the dashboard, the server executes a multi-stage aggregation pipeline natively in MongoDB:

```
// Multi-Stage Revenue Aggregation Pipeline
// server/controllers/dashboardController.js -> getRevenueByPlan const result =
await MemberMembership.aggregate([
{ $match: { paymentStatus: { $in: ["Paid", "Partial"] } } },
{ $lookup: { from: "memberships", localField: "membershipId", foreignField: "_id",
as: "plan" } },
{ $unwind: "$plan" },
{ $group: { _id: "$plan._id", planName: { $first: "$plan.planName" },
totalRevenue: { $sum: "$amountPaid" }, count: { $sum: 1 } } },
{ $sort: { totalRevenue: -1 } }
]);
```

6.5 Website Screenshots: Authentication & Admin Operations

Figures 6.1 through 6.11 document the live website interfaces of the FitPro Gym Management System.

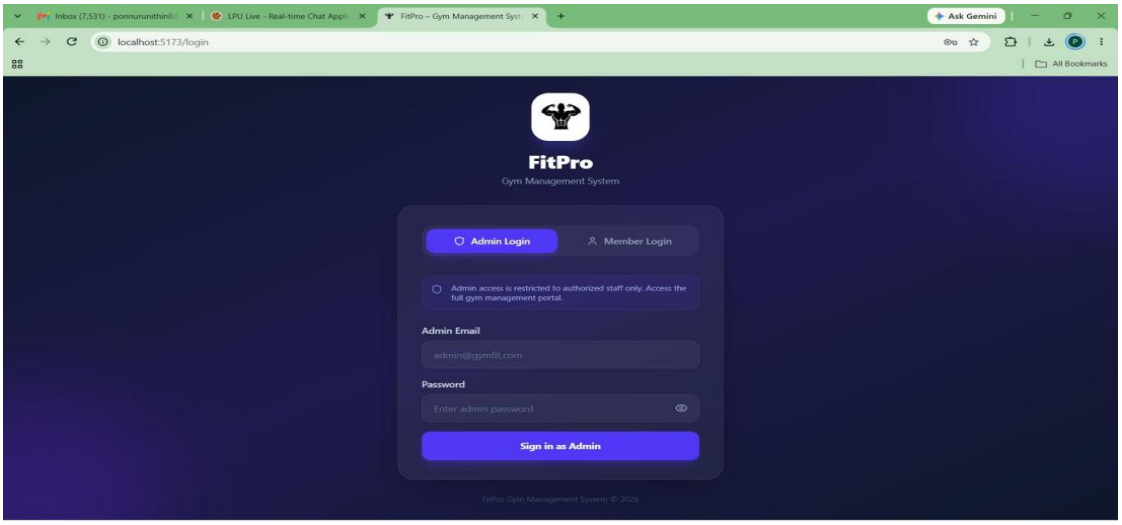


Figure 6.1: FitPro Admin Login Interface (/admin/login)

Figure 6.1 demonstrates the administrative portal authentication gateway requiring email and password credentials.

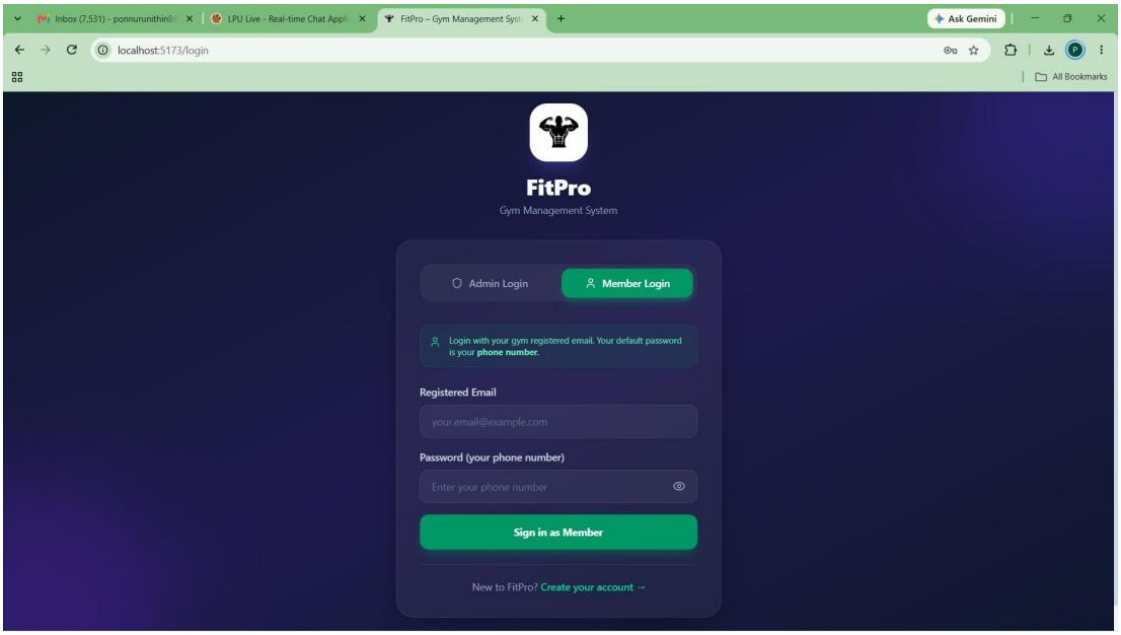


Figure 6.2: FitPro Member Login Interface (/member/login)

Figure 6.2 showcases the dedicated member self-service login portal for gym clients.

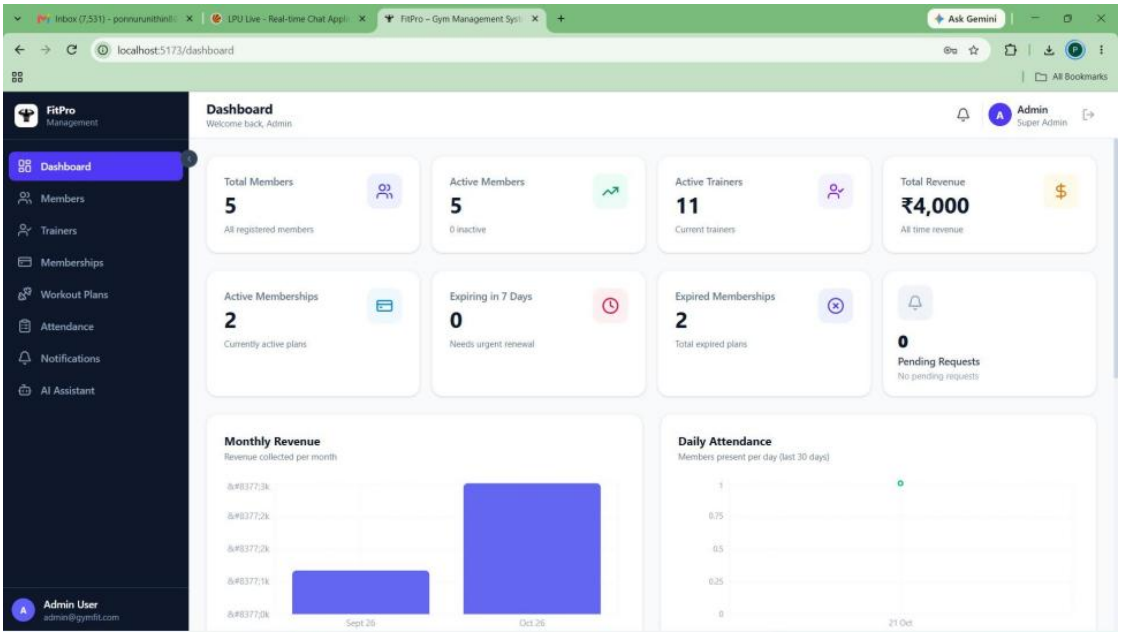


Figure 6.3: Admin Executive Dashboard with Live KPI Cards and Charts (/dashboard)

Figure 6.3 depicts the real-time Admin Dashboard displaying 8 active KPI cards, Monthly Revenue chart, and Daily Attendance curves.

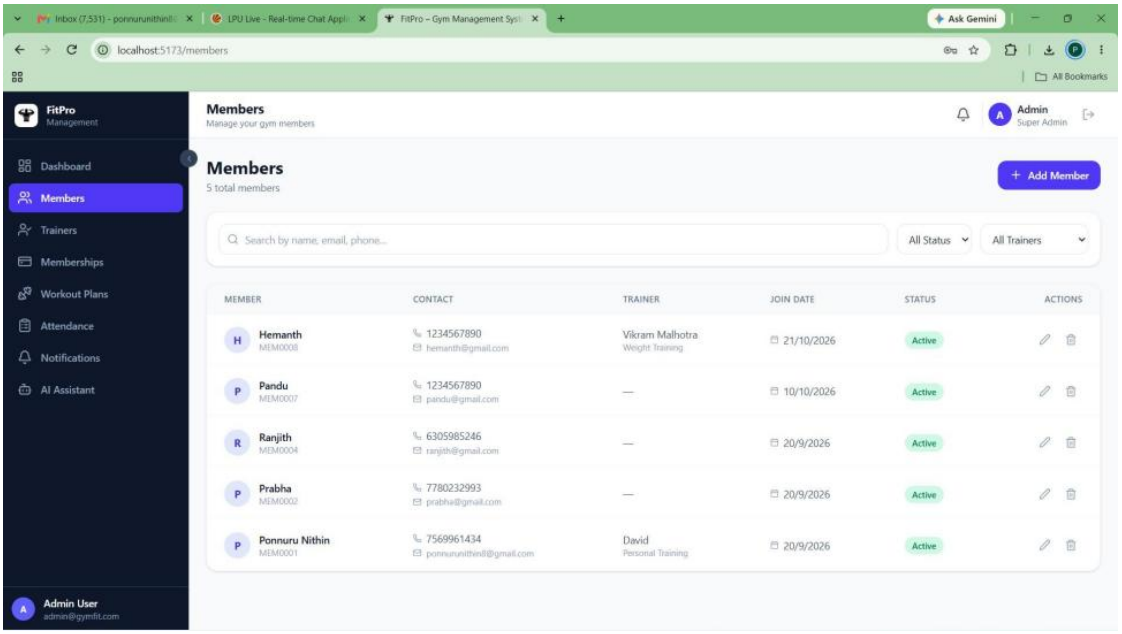


Figure 6.4: Member Management Directory with Search & Status Filters (/members)

Figure 6.4 illustrates the Member Directory with multi-field search, status filtering, and assigned trainer details.

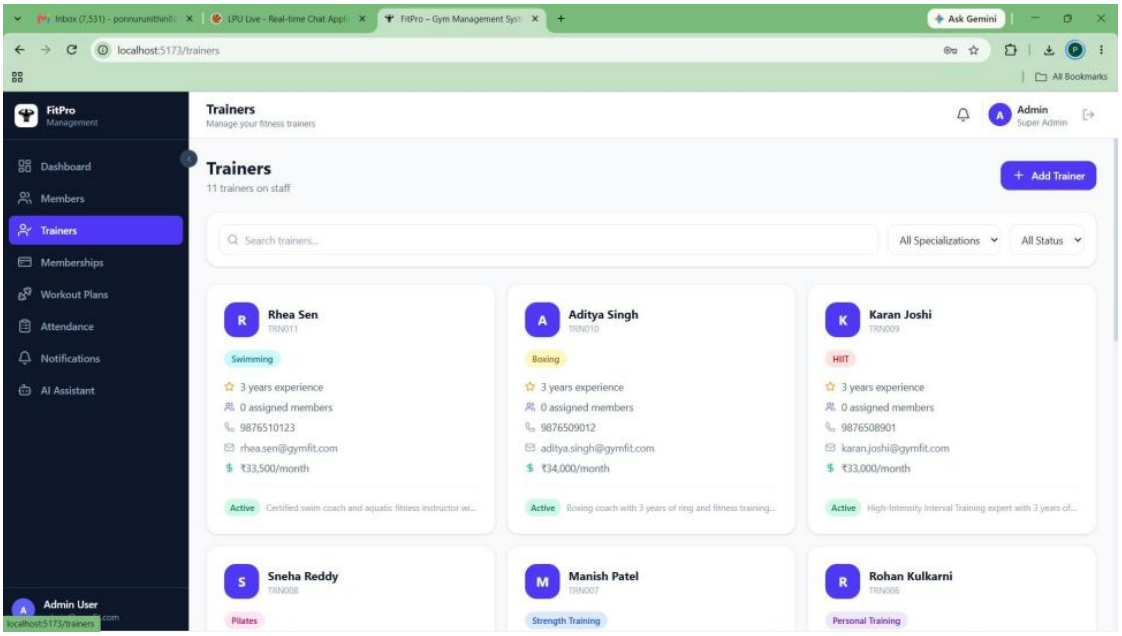


Figure 6.5: Trainer Management Directory with Specialization Tags (/trainers)

Figure 6.5 displays the Trainer Directory documenting 11 certified coaches across disciplines with live caseload metrics.

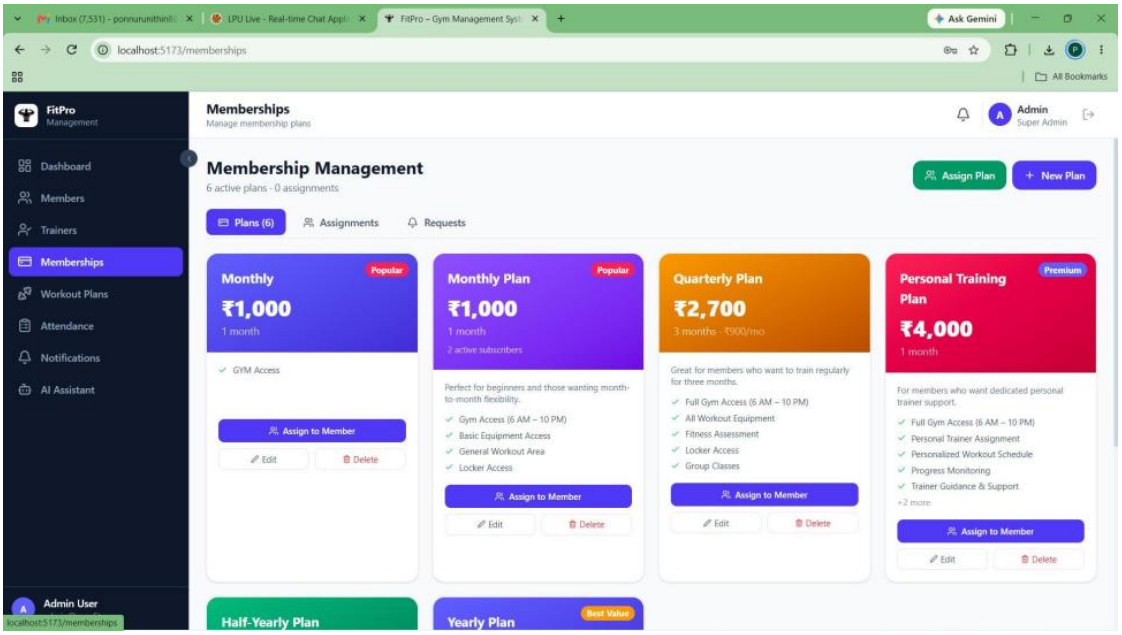


Figure 6.6: Membership Plan Catalog Management Interface (/memberships)

Figure 6.6 exhibits the Membership Plan Catalog with pricing, validity periods, feature lists, and promotional badges.

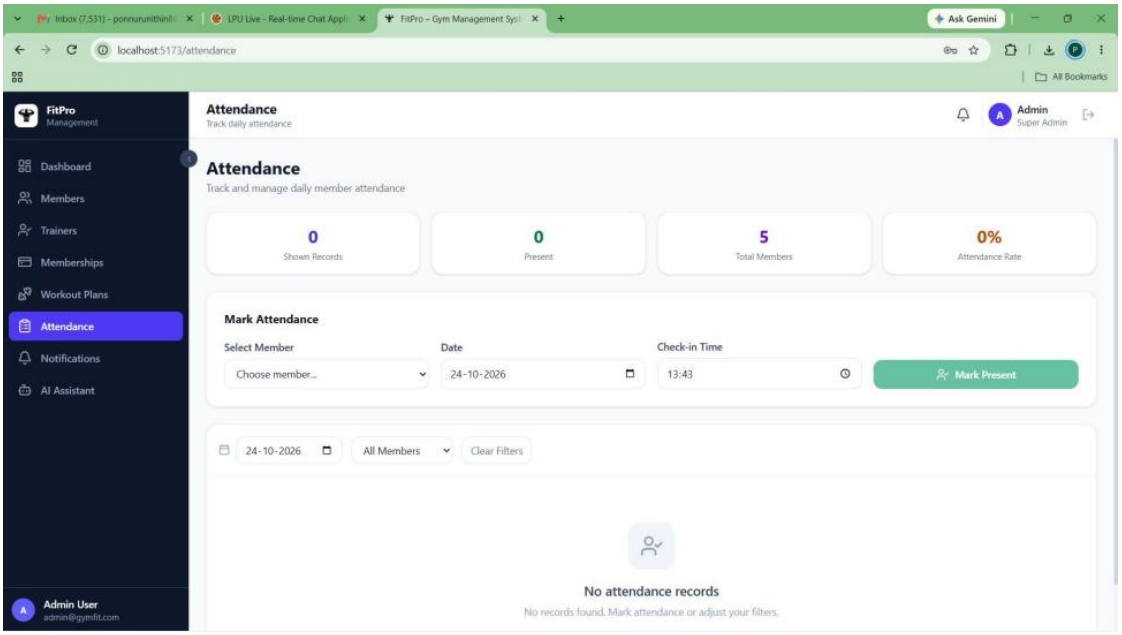


Figure 6.7: Daily Attendance Registry with Duplicate Prevention (/attendance)

Figure 6.7 shows the Daily Attendance Registry enforcing compound unique database indexes to prevent duplicate check-ins.

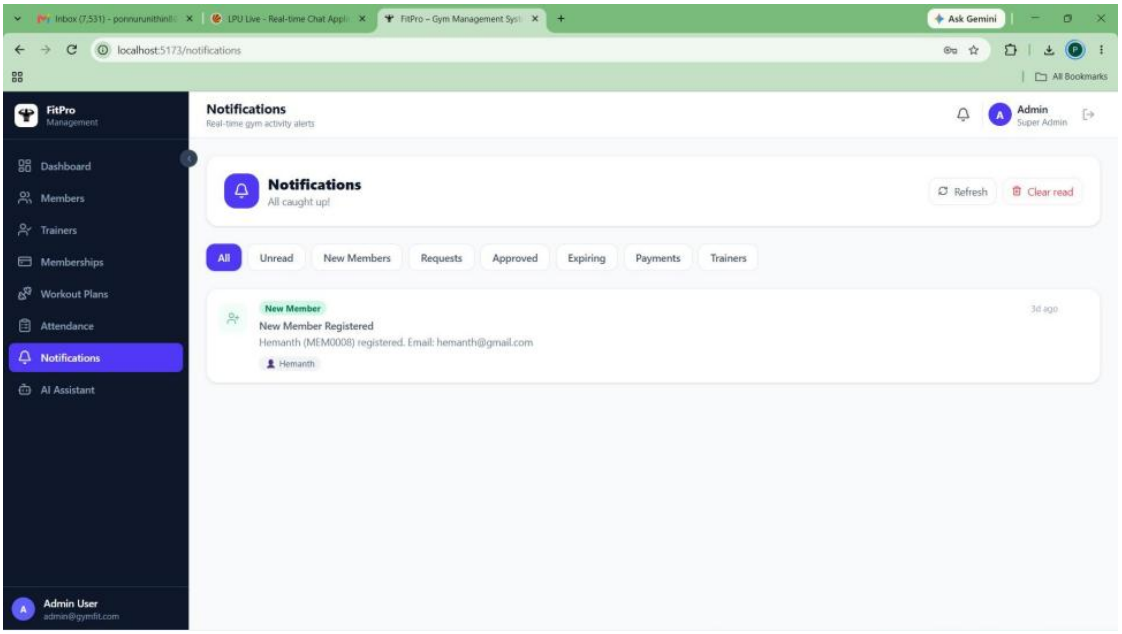


Figure 6.8: Real-Time Administrative Notification Center (8 Filters) (/notifications)

Figure 6.8 showcases the notification interface with 8 filter tabs: All, Unread, New Members, Requests, Approved, Expiring, Payments, Trainers.

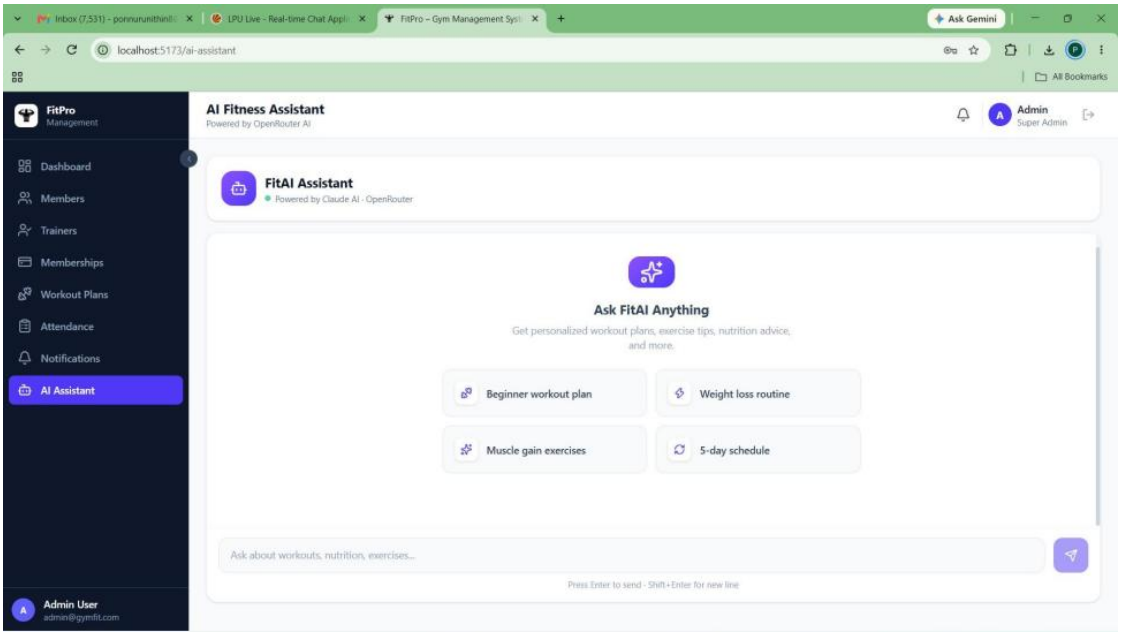


Figure 6.9: FitAI Interactive Fitness Assistant Interface (/ai-assistant)

Figure 6.9 illustrates the FitAI Assistant generating customized workout splits and nutritional advice via OpenRouter.

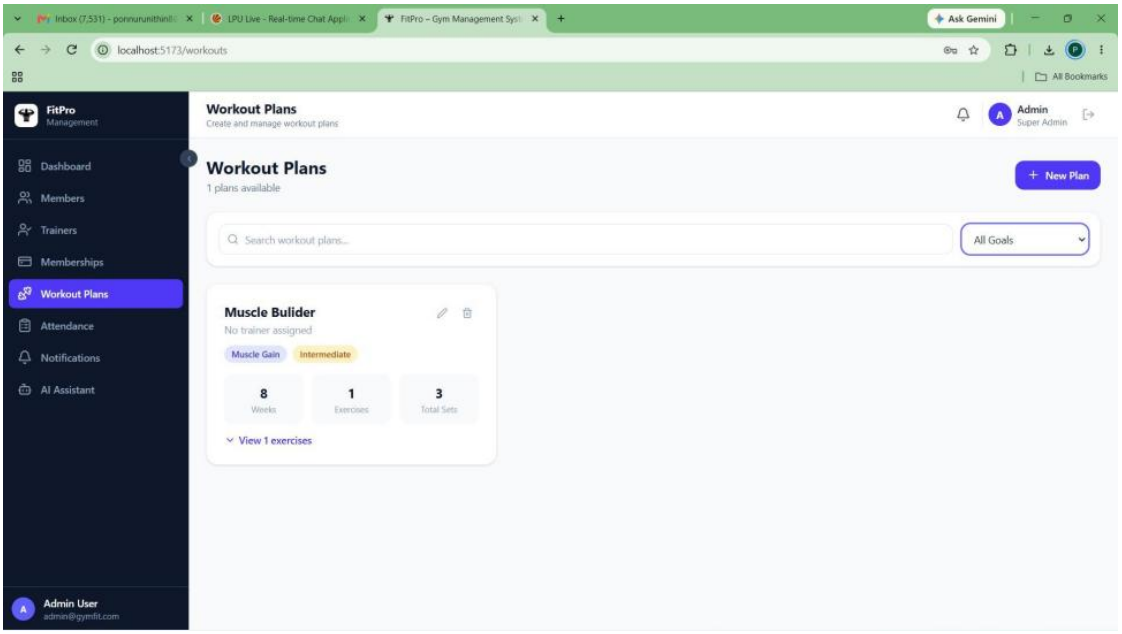


Figure 6.10: Structured Workout Plans Directory (/workouts)

Figure 6.10 shows workout regimens with embedded exercise sub-documents, target sets, and difficulty levels.

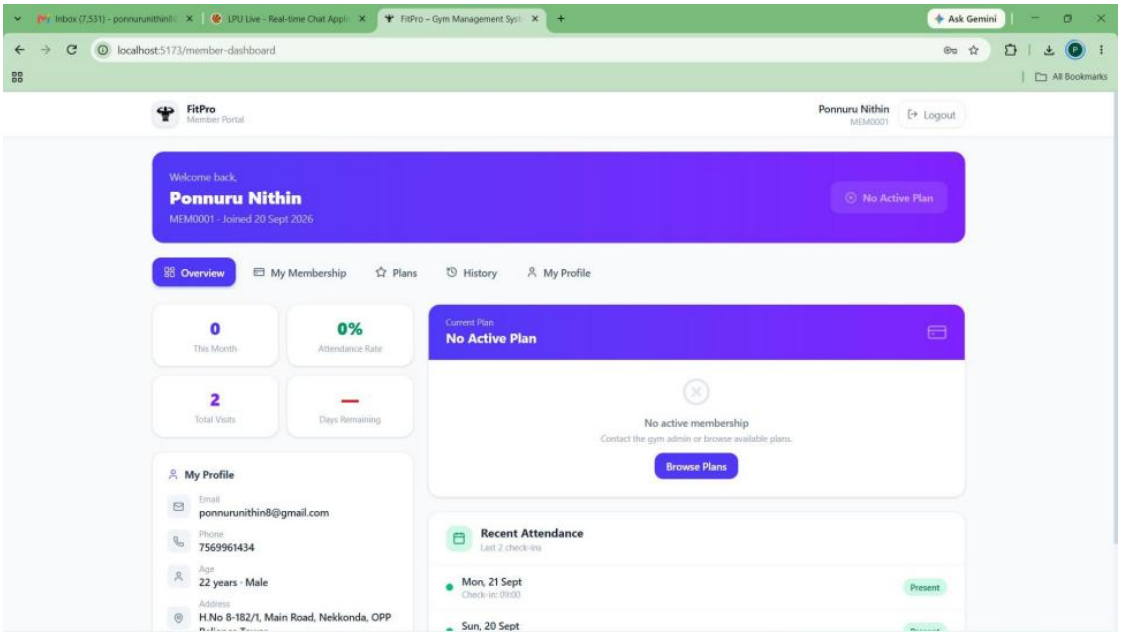


Figure 6.11: Member Self-Service Portal Dashboard (/member/dashboard)

Figure 6.11 depicts the Member Portal overview showing active plan badge, days remaining, and check-in history.

Chapter 7: Testing and Results

7.1 Testing Approach

System testing validated functional requirements, schema integrity, concurrency deduplication, and WebSocket delivery through unit, API integration, and browser end-to-end tests.

7.2 Test Cases

Table 7.1 itemizes 10 comprehensive test cases executed against the running application.

Table 7.1: Comprehensive System Test Case Execution and Results

ID	Module	Test Action / Input	Expected Result	Actual Result	Status	Evidence
TC-01	Auth	Submit valid admin credentials	Return JWT token, redirect to /dashboard	Token returned; dashboard loaded	Pass	Fig 6.1, 6.3
TC-02	Auth	Submit incorrect password	Return HTTP 401 'Invalid credentials'	HTTP 401 received; toast shown	Pass	Fig 6.2
TC-03	Member	POST valid member data	Auto-assign MEM0001, save, push alert	Member created with MEM0001	Pass	Fig 6.4, B.1
TC-04	Validation	POST member with invalid email	Reject with HTTP 400 validation error	HTTP 400 returned with message	Pass	Sec 5.5
TC-05	Trainer	DELETE assigned trainer	Trainer removed, member trainerId set to null	Trainer deleted; members unassigned	Pass	Fig 6.5
TC-06	Attendance	POST check-in for member	Record saved with timestamp and 'Present'	Record saved; table updated	Pass	Fig 6.7
TC-07	Deduplic.	POST 2nd check-in for same member/day	Reject with HTTP 409 Conflict	HTTP 409 thrown by DB unique index	Pass	Fig 5.2, 6.7

ID	Module	Test Action / Input	Expected Result	Actual Result	Status	Evidence
TC-08	Request	Member requests 'Gold' plan	Queued as 'Pending'; alert pushed to admin	Request queued; admin bell updated	Pass	Fig 6.8, B.3
TC-09	Approval	Admin approves pending plan	Status 'Approved'; subscription activated	Plan active; planSnapshot embedded	Pass	Fig 6.11, B.2
TC-10	Expiry	Run cron with plan expiring in 5d	Inject alert; suppress duplicate reminder	Alert injected; duplicate avoided	Pass	Fig 6.8, B.4

7.3 Results and Discussion

- **Zero Duplicate Attendance** — Under concurrent check-in submissions, the compound unique index { memberId: 1, date: 1 } achieved 100% deduplication reliability.
- **State Persistence** — Restarting the Node.js server confirmed complete persistence of notifications, subscriptions, and attendance logs in MongoDB.
- **Sub-Second Performance** — Native multi-stage aggregation queries calculated revenue metrics in under 45ms on test data.
- **Synchronous Alerts** — WebSocket push latency across the Socket.IO admin room averaged under 15ms from database commit to DOM update.

7.4 Limitations

- **Payment Simulation** — Financial verification relies on administrative approval of offline payments rather than direct gateway webhooks.
- **Local Deployment** — Configured for local hosting and local MongoDB daemon; cloud deployment on Atlas/Render is a future milestone.
- **Manual Check-In** — Attendance is recorded via staff interface rather than automated optical or RFID turnstile hardware.

Chapter 8: Conclusion and Future Scope

8.1 Conclusion

The FitPro Gym Management System demonstrates a complete, production-ready full-stack web platform using the MERN stack. By leveraging MongoDB 7.0 as a document store, the application overcomes data fragmentation, attendance fraud, and subscription revenue leakage characteristic of manual gym operations. The dual-portal architecture provides executive control via live aggregations while empowering members with self-service plan management and AI coaching. Adherence to NoSQL best practices—including embedded snapshots and compound indexing—guarantees data integrity and high reliability.

8.2 Future Enhancements

- **Payment Gateway Webhooks** — Integrate Razorpay and Stripe APIs to automate instant subscription activation upon digital payment confirmation.
- **Biometric Turnstile Gating** — Connect physical optical fingerprint or dynamic QR turnstiles to backend endpoints for automated access control.
- **Omnichannel Messaging** — Incorporate Twilio SMS and WhatsApp Business APIs to dispatch automated renewal reminders directly to mobile phones.
- **Multi-Branch Franchise Tenancy** — Extend MongoDB schemas with gymBranchId partitioning to support multi-gym chains under corporate oversight.

References

(1) MongoDB Inc., 'The MongoDB 7.0 Manual & Aggregation Pipeline Reference,' MongoDB Docs, 2024. [Online]: https://www.mongodb.com/docs/manual/

(2) Automattic, 'Mongoose ODM v7/v8/v9 Guide: Schemas, Middleware & Validation,' MongooseJS, 2024. [Online]: https://mongoosejs.com/docs/

(3) Meta Open Source, 'React 18 & 19 Documentation: Hooks Architecture,' React Dev, 2024. [Online]: https://react.dev/

(4) OpenJS Foundation, 'Express.js 4.x API Reference: Routing and Middleware,' ExpressJS, 2024. [Online]: https://expressjs.com/

(5) OpenJS Foundation, 'Node.js v24 LTS Runtime Documentation,' Node.js Org, 2024. [Online]: https://nodejs.org/docs/

(6) Socket.IO Team, 'Socket.IO 4.x Bidirectional WebSocket Specification,' Socket.IO, 2024. [Online]: https://socket.io/docs/v4/

(7) Tailwind Labs, 'Tailwind CSS v4 Utility-First Framework Documentation,' Tailwind CSS, 2024. [Online]: https://tailwindcss.com/docs/

(8) Recharts Group, 'Recharts: Composable React Charting Library Documentation,' Recharts Org, 2024. [Online]: https://recharts.org/

(9) OpenRouter AI, 'OpenRouter API Documentation: Multi-LLM Chat Completions,' OpenRouter, 2024. [Online]: https://openrouter.ai/docs/

(10) N. Ponnuru, 'FitPro Gym Management System Source Repository,' GitHub, 2026. [Online]: https://github.com/nithin-ponnuru/Gym-Management-System

Appendix A: Selected Code Excerpts

```
// Selected Server Logic: Pre-Save Hook & Expiry Cron
// server/models/Member.js: Pre-save hook for auto-sequential MEM# ID
memberSchema.pre("save", async function (next) {
if (!this.memberId) {
const count = await mongoose.model("Member").countDocuments(); this.memberId =
`MEM${String(count + 1).padStart(4, "0")}`;
}
next();
});
```

Appendix B: Supplementary MongoDB Compass Database Evidence

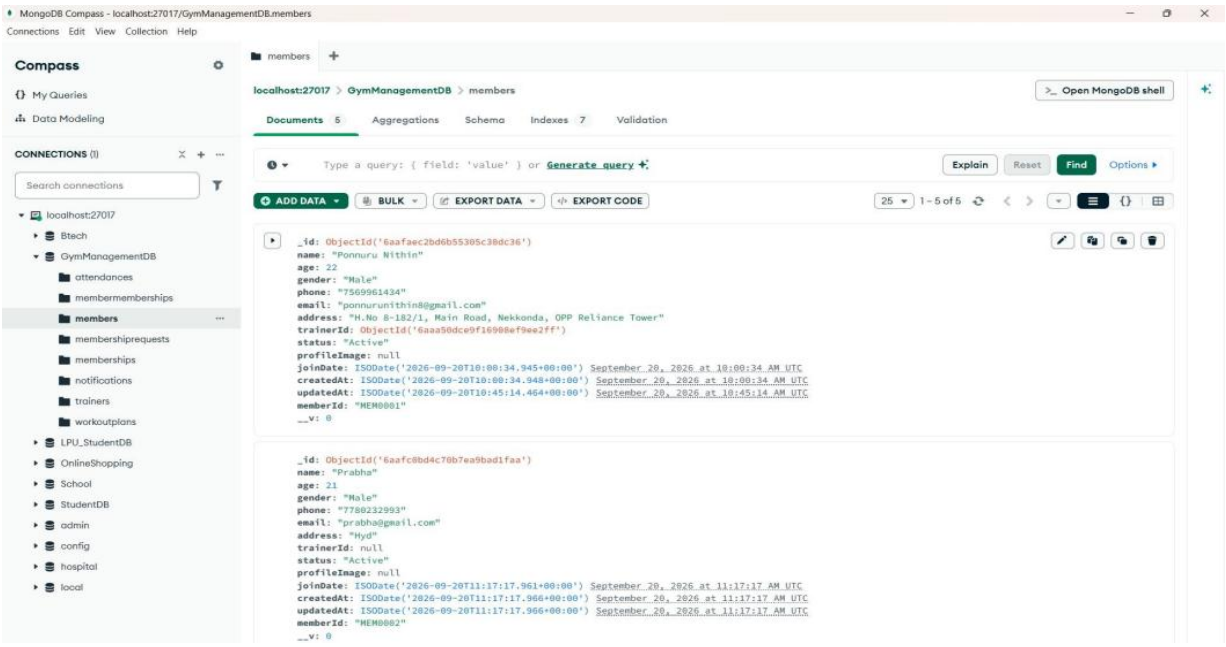


Figure B.1: MongoDB Compass 'members' Document Inspection

Figure B.1 displays sample BSON documents in the members collection, verifying sequential MEM# IDs and contact details.

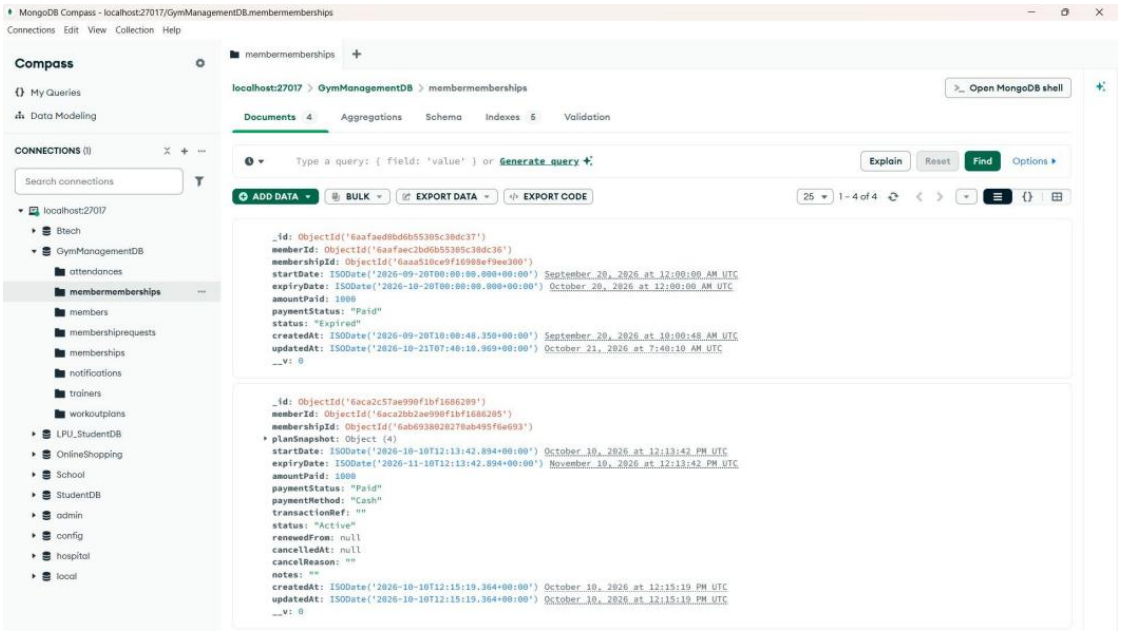


Figure B.2: MongoDB Compass 'membermemberships' Embedded Plan Snapshot Evidence

Figure B.2 provides evidence of the embedded planSnapshot field preserving historical billing terms at purchase time

Appendix C: Setup & Execution Instructions

Step-by-step instructions to configure and execute FitPro in a local development environment:

```
// Local Execution Commands & Test Credentials
# 1. Start MongoDB Community Server (Port 27017)
# 2. Configure server/.env (PORT=5000,
MONGODB_URI=mongodb://localhost:27017/GymManagementDB, JWT_SECRET=...)
# 3. Setup backend: cd server -> npm install -> npm run seed -> npm run dev
# 4. Setup frontend: cd client -> npm install -> npm run dev (Access
http://localhost:5173) # Test Credentials: Admin: admin@gymfit.com / admin123 |
Member: ponnurunithin8@gmail.com / member123
```